HOW-TO

Playwright generate PDF from HTML: script, options, checks

Updated 8 Dec 2026

KEY TAKEAWAYS

- Launch Chromium, load the page with `page.goto()` or `page.setContent()`, then call `page.pdf({ path, format: 'A4' })`.
- `format` defaults to Letter, every `margin` to 0 and `printBackground` to false.
- Header and footer are off unless you set `displayHeaderFooter: true`; the templates fill in classes such as `pageNumber` and `totalPages`.
- Wait for `document.fonts.ready` before printing, so late web fonts don't change where pages break.
- Count the pages in the finished PDF and compare them with the pages you expect before you keep the file.

CONTENTS

1 A Playwright script to generate a PDF from HTML 2
2 Playwright PDF options that matter 3
3 Headers, footers, or none 4
4 Waiting for the page 5
5 Checking the PDF before you keep it 5
6 Puppeteer, in short 6
7 How papyrus handles it 6
8 What to remember 7

To have Playwright generate a PDF from HTML, launch Chromium, open the page with `page.goto()` or `page.setContent()`, and call `page.pdf({ path: "out.pdf", format: "A4" })`. `page.pdf()` works only in Chromium: in Firefox or WebKit, Playwright throws “PDF generation is only supported for Headless Chromium”. You get the page printed with its print CSS, saved to the path you gave.

That’s enough for a first PDF. Making it repeatable in CI takes a few more things: the options that matter, headers and footers, waiting for fonts, and a check on the page count. This article assumes Node and a page you can open by URL, for example from a local server. For the CSS side, see css page size a4 for sizing and css page break print for breaks.

1. A Playwright script to generate a PDF from HTML

Install the library (not the test runner) and download Chromium:

```
npm i -D playwright pdf-lib
npx playwright install chromium
```

On a CI machine, Playwright’s browsers page uses `npx playwright install --with-deps chromium`, which installs the system dependencies as well. Then save this as `make-pdf.mjs`:

```
// node make-pdf.mjs http://localhost:3000/report report.pdf
import { readFile } from "node:fs/promises";
import { chromium } from "playwright";
import { PDFDocument } from "pdf-lib";

const [url, out = "out.pdf"] = process.argv.slice(2);

const browser = await chromium.launch();
try {
  const page = await browser.newPage();
  await page.goto(url);
  await page.evaluate(async () => { await document.fonts.ready; });

  const sheets = await page.locator(".sheet").count();
  await page.pdf({ path: out, format: "A4", printBackground: true });

  const pdf = await PDFDocument.load(await readFile(out));
  const pages = pdf.getPageCount();
  if (pages !== sheets) throw new Error(`${out}: ${pages} pages, ${sheets}
sheets`);
  console.log(`${out}: ${pages} pages`);
} finally {
  await browser.close();
}
```

It opens the URL, waits for fonts, counts the elements with the class `sheet`, prints, reads the PDF back and exits with an error if the counts differ. We ran it against a page of three A4 sheets and got a three-page PDF. On a two-sheet page whose sheets were 3 mm too tall, it stopped with “4 pages, 2 sheets”. If you only want the file, drop the `pdf-lib` lines and the `.sheet` count.

If you have an HTML string and no URL, call `await page.setContent(html)` instead of `goto`. Playwright’s docs say `setContent` calls `document.write()` internally and inherits its behaviour, which is one reason to prefer `goto` for a real page.

2. Playwright PDF options that matter

These are the `page.pdf()` options most PDF jobs touch, with the defaults Playwright’s docs list today:

Option	Default	What it does
`path`	none	Where to save the PDF. Without it nothing is written; the Buffer is still returned.
`format`	`'Letter'`	Paper name such as `'A4'` (8.27 × 11.7 in). Takes priority over `width` and `height`.
`width`, `height`	none	A number or a string with a unit. Plain numbers are pixels; units are `px`, `in`, `cm`, `mm`.
`margin`	0 on each side	An object with `top`, `right`, `bottom` and `left`.
`printBackground`	`false`	Print background graphics.
`preferCSSPageSize`	`false`	Give an `@page` size priority over `format`, `width` and `height`. When false, content is scaled to fit the paper.
`landscape`	`false`	Paper orientation.
`scale`	`1`	Rendering scale, between 0.1 and 2.
`pageRanges`	`''`	Pages to print, such as `'1-5, 8, 11-13'`. Empty means all.

The list also has `outline` and `tagged` (both default to false), and the header and footer options below.

Three defaults catch people out. The paper is Letter, so ask for `A4` if you want it. Backgrounds are off until you set `printBackground`. And `page.pdf()` prints with the `print` CSS media; for the screen styles, call `page.emulateMedia({ media: "screen" })` first. The docs also say colours are modified for printing by default, and point to `-webkit-print-color-adjust` to force exact colours.

Set the paper size in one place. If your CSS already has `@page { size: A4 }`, which the A4 page size article explains, pass `preferCSSPageSize: true` and the CSS wins.

---

3. Headers, footers, or none

By default Playwright prints no header and no footer, because `displayHeaderFooter` is false. That's the whole answer to "headless Chrome print to PDF with no header or footer" in Playwright: leave the option out.

To print page numbers, turn it on and pass templates. The docs say the header template should be valid HTML that uses the classes `date`, `title`, `url`, `pageNumber` and `totalPages` to inject values, and that the footer template uses the same format. Script tags inside the templates aren't run, and the page's styles aren't visible there, so style the template inline:

```
await page.pdf({
  path: "report.pdf",
  format: "A4",
  margin: { top: "20mm", bottom: "20mm" },
  displayHeaderFooter: true,
  headerTemplate: "<div></div>",
  footerTemplate: `<div style="font-size:9px;width:100%;text-align:center">
    <span class="pageNumber"></span> / <span class="totalPages"></span></div>`,
});
```

The example passes an empty header so the footer is the only thing printed, and leaves 20 mm top and bottom margins. Playwright's reference doesn't describe how the templates are placed on the page, so check the PDF. If CSS page-margin boxes are enough for you, the page-break article shows `counter(page)` and its limited browser support.

4. Waiting for the page

`page.goto()` and `page.setContent()` wait for the `load` event by default. Their `waitUntil` option also takes `domcontentloaded`, `networkidle` and `commit`, and the docs mark `networkidle` as discouraged. `page.waitForLoadState()` waits for one of those states after the fact.

`load` doesn't tell you that web fonts are ready. MDN says `document.fonts.ready` resolves once the document has finished loading fonts and layout is done. `page.evaluate()` waits for a returned Promise or an async function, so this one line covers it:

```
await page.evaluate(async () => { await document.fonts.ready; });
```

It matters because a font that arrives late changes the width of the text, lines wrap differently, and content moves between pages. papyrus's README gives the same reason for awaiting fonts before printing: so pagination doesn't depend on load timing. Puppeteer's PDF guide says its `Page.pdf()` waits for fonts by default.

The headless mode matters too. Playwright's browsers page says headless runs use a separate Chromium headless shell by default, while Chrome, Edge and the `chromium` channel use a newer headless mode, and that you should "expect different behavior in some cases". Pick one browser setup and compare PDFs only with output from the same one.

5. Checking the PDF before you keep it

A PDF that came out at 7 pages instead of 6 is the kind of bug nobody notices until it has been sent, and a Playwright job that generates a PDF from HTML will write it without complaint. So count the pages.

The script does that in two steps. `page.locator(".sheet").count()` returns the number of matching elements, which is your expected page count when each printed page is one `.sheet` element. Then pdf-lib, which its README says runs in Node, loads the file: `PDFDocument.load()` takes the bytes from `fs.readFile()`, and `getPageCount()` returns the number of pages in the document. Any PDF reader would do; what matters is that the number comes from the finished file.

If your page elements aren't one per printed page, hard-code the number you expect, or read it from a `data-` attribute your page sets.

6. Puppeteer, in short

Puppeteer does the same job with a different API. Its home page says it controls Chrome or Firefox over the DevTools Protocol or WebDriver BiDi, headless by default, and its `Page.pdf()` also prints with the `print` CSS media type. Neither project's PDF pages say anything about page-break rules, so a Puppeteer PDF page break is a CSS question either way; the page-break article covers it.

	Playwright	Puppeteer
Call	`page.pdf(options)`, returns a Buffer	`page.pdf(options?)`, returns a `Uint8Array`
`format` default	`'Letter'`	`letter`
`margin` default	0 on each side	no margins set
Waiting for fonts	no option; wait yourself	`waitForFonts`, default `true`
`timeout` option	none listed	`30_000` ms
`tagged` default	`false`	`true` (marked experimental)
Browsers it drives	Chromium, Firefox, WebKit (`page.pdf()` works only in Chromium)	Chrome, Firefox

Use the one you already run for tests. Both print with the page's print CSS, so the same stylesheet gives the same layout rules. The differences are in the options and their defaults, as the table shows. If you move a script from one to the other, check the format and the margins first, then fonts, since only Puppeteer waits for them on its own.

7. How papyrus handles it

papyrus is a toolkit by n-uf for documents rendered as HTML and exported as PDF, and its exporter does the steps above with headless Chromium through Playwright. papyrus and n-uf are not affiliated with Microsoft (Playwright), Google (Puppeteer, Chrome) or the Chromium project.

The command is `papyrus pdf --config papyrus.config.json`. For plain HTML files, point `baseUrl` at a folder and no server is needed:

```
{
  "baseUrl": "file:///path/to/site/",
  "docUrl": "{id}.html",
  "outDir": "dist",
  "docs": [{ "id": "report", "pageSelector": ".sheet", "pageCount": 3 }]
}
```

For an app, the server that renders the documents has to be running first; `--wait <seconds>` gives it time to answer, and `--port <n>` is a shorthand for a localhost base URL. `--only <id>` exports a single document. `sheet` is A4 by default and can be `"letter"` for the whole config or per document. `pageSelector` names the page elements when they aren't `.pdf-page`. Web fonts are awaited before printing, for at most 10 seconds.

Before writing each PDF, it checks that the number of page elements, the PDF's page count and the expected `pageCount` agree, so a broken render is never saved. `papyrus pdf --check` reports committed PDFs whose sources changed since they were printed, without launching a browser. `PAPYRUS_CHROMIUM_PATH` makes it print with an installed Chrome or Chromium instead of Playwright's download. From code, `renderPdf(browser, url, { sheet: "letter", pageSelector: ".page" })` takes the same options. `playwright` is an optional peer dependency, needed only for the export.

papyrus is young. Its versions are dates (`YY.M.R`), not semantic versions, so any release may break things. Pin an exact version and read the CHANGELOG before you upgrade. The product page, **html to pdf a4 with papyrus**, has the rest.

8. What to remember

- To make Playwright generate a PDF from HTML: `chromium.launch()`, `page.goto()`, `page.pdf({ format: "A4" })`. `page.pdf()` works only in Chromium.
- The defaults are Letter, no margins and no backgrounds.
- No header or footer unless `displayHeaderFooter` is true.
- Wait for `document.fonts.ready`, then count the pages in the finished file.

Questions people ask

How do I generate a PDF from HTML with Playwright?

Launch Chromium, open the page with `page.goto(url)` or `page.setContent(html)`, then call `page.pdf({ path: 'out.pdf', format: 'A4' })`. `page.pdf()` prints with the print CSS media and returns the PDF as a Buffer. Without `path` it doesn't save a file.

Does Playwright generate PDFs in Firefox or WebKit?

No. `page.pdf()` works only in Chromium. The `page.pdf()` reference doesn't list browsers, but Playwright's own code throws 'PDF generation is only supported for Headless Chromium' in the others, and its MCP PDF tool page says the same.

How do I print with no header and footer?

Leave `displayHeaderFooter` out. It defaults to false, so Playwright prints neither. Set it to true only when you also pass `headerTemplate` and `footerTemplate`.

How do I get A4 and margins in Playwright?

Pass `format: 'A4'` and a `margin` object, for example `margin: { top: '20mm', bottom: '20mm' }`. The default format is Letter and every margin defaults to 0. If your CSS sets an `@page` size, pass `preferCSSPageSize: true` so that size wins over `format`.

Why does the PDF break pages differently from run to run?

Often because a web font finished loading after layout, which changes text widths and line wraps. Wait for it with `await page.evaluate(async () => { await document.fonts.ready; })` before `page.pdf()`; MDN says that promise resolves once fonts have loaded and layout is done.

Sources

1. Playwright: Page (page.pdf, page.goto, page.setContent, page.waitForLoadState, page.emulateMedia) ↗ checked 11 Nov 2026
2. Playwright source at v1.58.2: pageDispatcher.ts ↗ checked 11 Nov 2026
3. Playwright: PDF Export (MCP tool) ↗ checked 11 Nov 2026
4. Playwright: Library ↗ checked 11 Nov 2026
5. Playwright: Browsers ↗ checked 11 Nov 2026
6. Playwright: Evaluating JavaScript ↗ checked 11 Nov 2026
7. Playwright: Locator ↗ checked 11 Nov 2026
8. MDN: FontFaceSet.ready ↗ checked 11 Nov 2026
9. pdf-lib README ↗ checked 11 Nov 2026
10. pdf-lib API: PDFDocument ↗ checked 11 Nov 2026
11. Puppeteer: home ↗ checked 11 Nov 2026
12. Puppeteer: PDF generation guide ↗ checked 11 Nov 2026
13. Puppeteer: PDFOptions ↗ checked 11 Nov 2026
14. Puppeteer: Page.pdf() ↗ checked 11 Nov 2026
15. papyrus README (@n-uf/papyrus v26.11.1) ↗ checked 11 Nov 2026
16. papyrus CLI help (@n-uf/papyrus v26.11.1, src/pdf/cli.js) ↗ checked 11 Nov 2026
17. papyrus CHANGELOG (@n-uf/papyrus v26.11.1) ↗ checked 11 Nov 2026